WEB-SCALE MULTIMEDIA DATA MANAGEMENT: CHALLENGES AND REMEDIES

Edward Y. Chang
edchang@google.com

Google Research

ABSTRACT

Text-based search engines have been flourishing. However, despite increasing needs of tools for organizing and searching imagery and video content, we have not seen a successful deployment of a Web-scale multimedia search engine. This talk (and paper) analyzes main technical challenges of such a deployment, and presents remedies in three areas: feature extraction, similarity characterization, and scalability.

1. INTRODUCTION

With rapid growth of personal and Web imagery and video data, it is increasingly crucial to provide tools that can assist with effective organization and search. Though advances have been made in image/video processing, indexing, and statistical learning, challenges remain for the deployment of a Web-scale multimedia search engine. In this paper, we present three major challenges and potential remedies.

The first challenge is extracting useful features to represent imagery and video data. Good features must be extracted from data so that the mapping between raw data and semantics can be effectively performed. We discuss the state-of-the-art features, their shortcomings, and remedies in Section 2.

The second challenge is similarity measure. Accounting accurately for perceptual similarity is difficult, partly because similarity is subjective and partly because features extracted from similar media can be quite different. For instance, textures of a down-sampled image can be quite different from those of the original image. Videos of the same scene taken in different lighting conditions can display different colors. We discuss state-of-the-art work in distance-function formulation in Section 3.

The third challenge that hinders the deployment of a large-scale system is scalability. A multimedia search engine must be able to scale well with respect to both data dimensionality and data quantity. In Section 4, we present recent advances in large-scale statistical learning, indexing, and searching.

2. FEATURE EXTRACTION

It is easier for human to identify key features in a photo than to identify key features in a text document. (Preschool kids can tell objects in a photo but cannot easily tell key phrases that make up a text article.) On the contrary, automatic feature extraction from a text article (using computer algorithms) is easy, but doing so from a photo is hard. Features of a text article such as words, n-grams, and some combinations of words can be fairly good in representing the article. Reliably extracting objects from a photo, however, is not technically attainable. Though computer-vision research has made marked progress, it is commonly agreed upon that the deployment of a robust object segmenter is still decades away.

Given the fact that accurate and efficient object segmentation is yet to be feasible, most studies and systems resort to extracting various *holistic* and *local* features. Holistic features of color, texture, and shape make sense if we divide human vision into the perceiving part and cognitive part [6]. The perceiving part simply senses photon patterns, which are pixel groups of color, texture, and shape. The cognitive part then performs semantic analysis on the perceived patterns. Unfortunately, even extracting reliable, holistic features is technically difficult. Our experiment conducted in a video-surveillance setting [27] shows that a shirt can look in different colors under different lighting conditions (confirming results of many prior studies). And even given the same lighting condition, different cameras can capture a shirt in different colors. This problem is widely known as *color constancy* and there is no robust solution in sight. Regarding texture features, the same object seen in different depths can produce signals of different granularities. Similarly, a compressed photo, thought looks perceptually identical to the original one, can produce different texture signatures. The same scene or the same object can produce different holistic features. This "aliasing" effect in perceptual features makes similarity measure difficult.

Regarding *local* features, Scale Invariant Feature Transform (SIFT) is an effective representation [15]. SIFT extracts local, distinctive, invariant features. SIFT consists of four major stages: 1) scale-space extreme detection,

2) keypoint localization, 3) orientation assignment, and 4) keypoint descriptor. Features generated by SIFT have been shown to be robust in matching across an affined distortion, change in 3D viewpoint, addition of noise, and change of illumination. SIFT is useful for recognizing objects with rigid shapes.

The key take-away from this feature-extraction discussion is that representing imagery and video data cannot be performed as well as representing text data. Nevertheless, this fact does not entirely deter developing an effective image or video search engine. Noisy perceived features put heavy burden on the cognitive part of the system to mask errors. Statistical methods presented in the next two sections are the key to the success of a media data management system.

Perceptual features discussed this far cannot provide support to all queries [30]. For instance, one cannot tell time or location from a photo. We have to resort to other meta data such as camera parameters and user logs to provide comprehensive coverage of semantics. Semantic labels (queries) can be roughly divided into two categories: wh labels and non-wh labels. Wh-semantics include time (when), people (who), location (where), landmarks (what), and event (inferred from when, who, where, and what). Perceptual features cannot infer these wh-semantics. Fortunately, already cameras can provide time, and we can easily infer location from GPS or CellID. Determining the what and who requires annotation, which can be obtained indirectly from user logs. For instance, a user queries "Modena Italy" and clicks on photos matching the query. The label "Modena Italy" can be assigned to the clicked photos. With a sufficiently large user base and a large number of queries, such label association can be done quite reliably.

In summary, at least three short-term research challenges remain in improving media-data representation: 1) how to bootstrap a search engine when user log does not yet exist, 2) how to remove noise (spams) from the log, and 3) how to perform multimodal (e.g., perceptual features, camera parameters, and user logs) integration both effectively and efficiently [29].

3. SIMILARITY CHARACTERIZATION

At the heart of information management and retrieval tasks is a distance function that measures *similarity* between data instances. To date, most applications employ a variant of the *Euclidean distance* for measuring similarity. However, to measure similarity meaningfully, an effective distance function ought to consider the idiosyncrasies of the application, data, and user (hereafter we refer to these factors as contextual information). The quality of the distance function significantly affects the success in organizing data or finding meaningful results [1, 2, 4, 8, 9].

We believe the best distance function for matching photos and videos to be dependent on semantics. The best similarity function for comparing faces is different from that for comparing buildings or flowers. How do we consider contextual information (semantics or users) in formulating a good distance function? One extension of the popular Euclidean distance (or more generally, the L_p-norm) is to weight the data attributes (features) based on their importance for a target task [2, 8]. For example, for answering an *ocean* image-query, color features should be weighted higher. For answering an *architecture* image-query, shape and texture features may be more important. Weighting these features is equivalent to performing a *linear* transformation in the space formed by the features. Although linear models enjoy the twin advantages of simplicity of description and efficiency of computation, this same simplicity is insufficient to model similarity for many real-world datasets. For example, it has been widely acknowledged in the image/video retrieval domain that a query concept is typically a nonlinear combination of perceptual features (color, texture, and shape) [21]. A more powerful strategy is to perform a *nonlinear* transformation on the feature space to gain greater flexibility for mapping features to semantics.

We proposed such an approach named DA$_{lign}$ or *distance-function alignment* in [26]. The inputs to DA$_{lign}$ are a prior distance function, and *contextual information*. Contextual information can be conveyed in the form of *training data*. For instance, in the information-retrieval domain, Web users can convey information via relevance feedback showing which photos are relevant to their queries. DA$_{lign}$ transforms the prior function to capture the nonlinear relationships among the *contextual information*. The similarity scores of unseen data-pairs can then be measured by the transformed function to better reflect the idiosyncrasies of the application, data, and user.

At first it might seem that capturing nonlinear relationships among contextual information can suffer from high computational complexity. DA$_{lign}$ avoids this concern by employing the *kernel trick* [3]. The kernel trick lets us generalize distance-based algorithms to operate in the *projected space* (defined next), usually nonlinearly related to the *input space*. The *input space* (denoted as $\mathcal{I}$) is the original space in which data vectors are located. The advantage of using the *kernel trick* is that, instead of explicitly determining the coordinates of the data vectors in the projected space, the distance computation in $\mathcal{P}$ can be efficiently performed in $\mathcal{I}$ through a kernel function. Specifically, given two vectors $\mathbf{x}_i$ and $\mathbf{x}_j$, kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$ is defined as the inner product of $\phi(\mathbf{x}_i)$ and $\phi(\mathbf{x}_j)$, where ϕ is a basis function that maps the vectors $\mathbf{x}_i$ and $\mathbf{x}_j$ from $\mathcal{I}$ to $\mathcal{P}$. The inner product between two vectors can be thought of as a measure of their similarity. Therefore, $K(\mathbf{x}_i, \mathbf{x}_j)$ returns the similarity of $\mathbf{x}_i$ and $\mathbf{x}_j$ in $\mathcal{P}$. The distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ in terms of the kernel is defined as

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\phi(\mathbf{x}_i) - \phi(\mathbf{x}_j)\|_2$$

$$= \sqrt{K(\mathbf{x}_i, \mathbf{x}_i) + K(\mathbf{x}_j, \mathbf{x}_j) - 2K(\mathbf{x}_i, \mathbf{x}_j)}. \quad (1)$$

Since a kernel function can be either linear or nonlinear, the traditional feature-weighting approach (e.g., [2]) is just a special case of $\mathsf{DA}_{\mathit{lign}}$.

One major challenge about statistical learning in a query scenario is the shortage of training data. As discussed in [10], the number of training instances is often less than the data dimensionality, and we call this *small sample-learning* problem (samples can also be imbalanced [24]). We proposed in [13] the Unified Learning Paradigm, which combines information that can be effectively collected from unlabeled data for augmenting information that can be collected from labeled data. In addition, when labeled data are collected, active learning is employed to maximize the usefulness of the collected information [21]. Together, supervised, unsupervised, active, and reinforcement learning work synergistically for formulating an effective distance function to measure similarity.

An important note on the distance function is that similarity measures that can uncover partial matches between objects are essential. Two objects can be considered semantically similar when some respects closely match, even when they are very different in other respects. Popular measures such as Euclidean distance are not well suited to uncovering such similarities. Some recent works [14, 22] have proposed distance measures that are based on partial match. Let us use an example to illustrate. Suppose we are asked to name two places that are similar to Modena. Among several possibilities, Florence and Rome could be two reasonable answers. However, the respects Modena is similar to Florence differ from those in which England is similar to Rome. If we use the shared attributes of Modena and Florence to compare Modena and Rome, the latter pair might not be similar, and vice versa. Murphy and Medin [17] provide early insights into how similarity works in human perception: "The explanatory work is on the level of determining which attributes will be selected, with similarity being at least as much a consequence as a cause of a concept coherence." Goldstone [11] explains that similarity is the process that determines the respects for measuring similarity. In other words, a distance function for measuring a pair of objects is formulated only after the objects are compared, not before the comparison is made. The respects for the comparison are activated in this formulation process. The activated respects are more likely to be those that can support coherence between the compared objects.

In summary, two immediate research challenges are: 1) how to effectively account for dynamic and partial similarity, and 2) how to learn a distance function in a context-dependent way. We have developed preliminary solutions, and challenges remain in making these methods scalable in both data dimensionality and data quantity. In the section that follows, we address the issue of scalability.

4. SCALABILITY

As mentioned in the previous section, the kernel methods have been validated to be very useful for distance-function learning, and in general, for machine-learning tasks such as classification and clustering. The kernel methods use the kernel trick to project data from their *input space* to a much higher dimensional *feature space*, where a linear solution can be found. Despite of their efficiency in comparison to traditional non-linear methods, the kernel methods are still resource intensive. When working with Support Vector Machines (SVMs) [23], for example, the memory use is $\mathcal{O}(n^2)$ and computational cost as high as $\mathcal{O}(n^3)$. In Section 4.1, we discuss our parallel, approximate approach [28, 25, 5] to speed up the kernel methods in the training or clustering phase.

Another challenging about employing the kernel methods is that after a classifier has been trained, data must be indexed to support efficient k-NN search. We discuss indexing issues in Section 4.2.

4.1. Speeding up Kernel Methods

We started our effort in 2005 in speeding up the kernel methods. Our target algorithms include SVMs, SVD, EM, and spectral clustering algorithm. In this section, we use SVMs to illustrate our solutions. To explain the resource problem of the kernel methods and our proposed solution, we examine the resource bottlenecks of SVMs in a binary classification setting. Given a set of training data $\mathcal{X} = \{(\mathbf{x}_i, y_i) | \mathbf{x}_i \in \mathbf{R}^d\}_{i=1}^n$, where $\mathbf{x}_i$ is an observation vector, $y_i \in \{-1, 1\}$ is the class label of $\mathbf{x}_i$, and n is the size of $\mathcal{X}$, we apply SVMs on $\mathcal{X}$ to train a binary classifier. The training of SVMs boils down to solving the following convex Quadratic Programming (QP) problem:

$$\begin{aligned} \min \quad & \mathcal{D}(\boldsymbol{\alpha}) = \frac{1}{2}\boldsymbol{\alpha}^T \mathbf{Q}\boldsymbol{\alpha} - \boldsymbol{\alpha}^T \mathbf{1} \\ s.t. \quad & \mathbf{0} \leq \boldsymbol{\alpha} \leq \mathbf{C},\ \mathbf{y}^T\boldsymbol{\alpha} = 0, \end{aligned} \quad (2)$$

where $[\mathbf{Q}]_{ij} = y_i y_j \boldsymbol{\phi}^T(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_j)$, and $\boldsymbol{\alpha} \in \mathbf{R}^n$ is the Lagrangian multiplier variable. The dual formulation $\mathcal{D}(\boldsymbol{\alpha})$ requires an inner product of $\boldsymbol{\phi}(\mathbf{x}_i)$ and $\boldsymbol{\phi}(\mathbf{x}_j)$. SVMs utilize the kernel trick by specifying a kernel function to define the inner-product $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}^T(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_j)$. This kernel trick eliminates the need to project instances $\boldsymbol{\phi}(\mathbf{x}_i)$ and $\boldsymbol{\phi}(\mathbf{x}_j)$ explicitly to a feature space. Instead, the dot product is performed in the d-dimensional input space. We thus can rewrite $[\mathbf{Q}]_{ij}$ as $y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$. This QP problem can be solved via the *Interior-Point method* [16], which memory usage is $\mathcal{O}(n^2)$ and computational cost $\mathcal{O}(n^3)$.

The reason for the quadratic memory use and cubic computational cost is because of the size of the kernel matrix Q. In [5], we present a parallel, row-based Incomplete

Cholesky Factorization (PICF) algorithm to distribute n data instances onto m machines. PICF loads the data in a round-robin fashion onto m machines. The memory requirement per machine is $\mathcal{O}(nd/m)$, where d is the size of each data instance. PICF then factorizes Q on m machines in parallel. At the end of PICF, each machine stores only a fraction of the factorized matrix, which takes up space of $\mathcal{O}(np/m)$, where p is the column dimension of the factorized matrix. In a large-scale problem where n is very large, d is typically much smaller than p and p is much smaller than n. PICF effectively reduces memory use from $\mathcal{O}(n^2)$ to $\mathcal{O}(np/m)$. For instance, on a one-million data set, storing $1M \times 1M \times 4$-Byte Q requires $4,000$GB storage. On 20 machines and setting $p = \sqrt{n}$, PICF can reduce the storage per machine to be 200MB.

ICF can approximate Q ($Q \in R^{n \times n}$) by a smaller matrix H ($H \in R^{n \times p}, p \ll n$), i.e., $Q \approx HH^T$. Parallel ICF, together with SMW (the *Sherman-Morrison-Woodbury formula*), can greatly reduce the memory use and computational complexity in solving an $n \times n$ linear system.

Let vector $\mathbf{v}$ be the diagonal of Q and suppose the pivots are $\{i_1, i_2, \ldots, i_k\}$, the kth iteration of ICF computes three equations:

$$H(i_k, k) = \sqrt{\mathbf{v}(i_k)} \quad (3)$$

$$H(J_k, k) = (Q(J_k, k) - \sum_{j=1}^{k-1} H(J_k, j)H(i_k, j))/H(i_k, k) \quad (4)$$

$$\mathbf{v}(J_k) = \mathbf{v}(J_k) - H(J_k, k)^2, \quad (5)$$

where J_k denotes the complement of $\{i_1, i_2, \ldots, i_k\}$. The algorithm iterates until the approximation of Q by $H_k H_k^T$ (measured by $trace(Q - H_k H_k^T)$) is satisfactory, or the predefined maximum iterations (or say, the desired rank of the ICF matrix) p is reached.

As suggested by Golub and Van Loan, a parallelized ICF algorithm can be obtained by constraining the parallelized Cholesky Factorization algorithm, iterating at most p times. Unfortunately, in the proposed algorithm [12], matrix H is distributed by columns in a round-robin way on m machines (hence we call it column-based parallelized ICF). Such column-based approach cannot gain full benefit from parallelization for two major reasons:

1. Large storage requirement. All data are needed for each machine to calculate $Q(J_k, k)$. Therefore, each machine must be able to store a local copy of the data.

2. Limited parallelizable computation. Only the inner product calculation ($\sum_{j=1}^{k-1} H(J_k, j)H(i_k, j)$) in (4) can be parallelized. The calculation of pivot selection, the summation of local inner product result, column calculation in (4), and the vector update in (5) must be performed on one single machine.

To remedy these shortcomings of the column-based approach, we propose a row-based approach to parallelize ICF. Our row-based approach starts by initializing variables and loading data onto m machines in a round-robin fashion. The algorithm then performs the ICF main loop until the termination criteria are satisfied (e.g., the rank of matrix H reaches p). In the main loop, PICF performs five tasks in each iteration k:

1. Distributedly find a pivot, which is the largest value in the diagonal $\mathbf{v}$ of matrix Q. Notice that PICP computes only needed elements in Q from data, and it does not store Q.

2. Set the machine where the pivot resides as the *master*.

3. On the *master*, PICF calculates $H(i_k, k)$ according to (3).

4. The *master* then broadcasts the pivot instance $\mathbf{x}_{i_k}$ and the pivot row $H(i_k, :)$.

5. Distributedly compute (4) and (5).

At the end of the algorithm, H is stored distributedly on m machines, ready for a linear solver. Use SVMs as an example, parallel IPM can readily be applied on the distributed data. PICF enjoys three advantages: parallel memory use ($\mathcal{O}(np/m)$), parallel computation ($\mathcal{O}(p^2n/m)$), and low communication overhead. Particularly on the communication overhead, its fraction of the entire computation time shrinks as the problem size grows. This pattern permits a larger problem to be solved on more machines to take advantage of parallel memory use and computation.

PICF is now being used by our parallel SVM (PSVM) algorithm to speed up training time from about $\mathcal{O}(n^2)$ to $\mathcal{O}(np^2/m)$. We have experimented PSVM [5] at Google on up to 250 machines and verified it to be effective. Our future work will focus on making PICF fail-safe. We are currently investigating a similar technique employed by [7] to achieve the fail-safe objective. At the same time, we are implementing parallel SVD, parallel EM, and a parallel version of spectral clustering algorithm. All these algorithms have been proven to be very useful for multimedia data management. Our work aims to provide a scalable infrasture for multimedia computing.

4.2. Indexing for Kernel Methods

Query-concept learning and *efficient retrieval* of relevant instances lie at the heart of multimedia retrieval, Support Vector Machines (SVMs), or more generally kernel methods, have made the *hyperplane query* a significant new paradigm for both *query-concept learning* and *instance retrieval*. The key advantage of a *hyperplane query* over a traditional *point query* is that the hyperplane query can be

richly defined with positive and negative data-instances, as compared to a single positive instance in the point query. A hyperplane query represents the query concept as a binary classifier separating the positive and negative instances.

In the *query-concept learning* phase, active learning is frequently employed to refine the user concept iteratively. In order to learn the target concept, the user is queried for the labels of a few instances at each iteration. The instances to be labeled by the user are chosen on the basis of the degree of uncertainty in their current classification so that the concept learning can achieve fast convergence. The query of interest in this phase is formulated as the set of top-k instances given the lowest absolute scores by the SVM classifier. These are the instances closest to the hyperplane. Having converged on the concept of interest, the *retrieval phase* of query processing returns the most relevant instances to the learned concept to the user. The top-k instances of interest with respect to the query concept are the k instances with the highest positive scores i.e., the k farthest data instances from the hyperplane in the relevant (positive) half-plane.

In [18, 20, 19], we propose a couple of efficient approaches to process top-k hyperplane queries, for both most uncertain and most relevant queries. Without an efficient pruning algorithm, processing top-k queries of both kinds requires sequentially scanning the entire dataset to find matching instances. When the number of instances is in the order of thousands or more, such naive sequential processing is simply not scalable. To be more specific about the problem at hand, we assume that Support Vector Machines (SVMs) is the employed learning algorithm to obtain the query hyperplane. SVMs find a maximum margin hyperplane separating the positive training instances from negative ones. SVMs can employ implicit projection to find a separating hyperplane (linear boundary) in a projected space $\mathcal{H}$. The linear boundary in $\mathcal{H}$, translates to a complex (often nonlinear) boundary in the input space, making SVMs a powerful tool for pattern recognition. The hyperplane is then used to obtain instance scores. The score of an instance is its distance from the hyperplane in the projected space $\mathcal{H}$. The SVM algorithm determines the top-k instances (nearest to or farthest from the hyperplane) in the corpus by computing the scores of *all* the instances. The top-k instances with the lowest absolute scores are returned when the most ambiguous samples are desired and the top-k instances with highest positive scores are selected when the most relevant instances are queried. Given the usually large dataset sizes, evaluation of the scores of all instances (in either the concept-learning or the instance-retrieval phase) involves multiple disk accesses and can quickly become extremely costly. Our proposed pruning approach speeds up retrieval of the top-k ambiguous and the top-k positive instances identified by SVMs. Our approach encloses the regions containing relevant instances in bounding boxes. Only the scores of instances within the bounding boxes are evaluated in our quest for the top-k instances.

In summary, scalability in both the training and retrieval phase is critical to make Web-scale deployment feasible. In the Web era, an algorithm that cannot take advantage of a large number of machines running in parallel might be scientific interesting but practically useless. We challenge the community to consider scalability as a key factor in designing a multimedia search engine.

5. CONCLUSION

In this paper, we have discussed three main challenges to the deployment of a large-scale multimedia information retrieval system: feature extraction, similarity characterization, and scalability. We argued that improving feature extraction can be hard in the near future. A system should take advantage of additional meta data made available by digital cameras and in user logs. When features are noisy, statistical methods are essential for improving the performance of query-concept learning, distance-function learning, and clustering. We thus presented statistical methods for effectively quantifying similarity between media data. Our methods offer two distinctive features. First, we employ the kernel trick to learn non-linear combinations of features, which is more powerful than traditional linear methods in adapting to contextual information. We show that the kernel trick does not incur additional cost in learning a distance function. Second, our distance-function learning takes advantage of unlabeled data, as well as most informative labeled data, through a unified learning paradigm. Finally, to ensure scalability of various tasks of multimedia information management and retrieval, we presented our work in approximate and parallel matrix factorization and computations. Through SVMs, we showed the efficiency of PICF, which can significantly improve both memory use and computation speed, and hence can scale the amount of data processed to the billion scale.

The other important area of topic is to integrate text with media data. One integration point is to ask users to annotate some photos/videos, and propagate them to other photos/videos. Another integration is to consider user logs and assigned the query terms to clicked photos/videos. We believe that multimodal integration will also find scalability an important issue, and our scalable infrastructure useful.

6. REFERENCES

[1] C. Aggarwal, A. Hinneburg, and D. Keim. On the surprising behavior of distance metrics in high dimensional spaces. In *Proceedings of International Conference on Database Theory*, pages 420–434, 2001.

[2] C. C. Aggarwal. Towards systematic design of distance functions for data mining applications. *The Ninth ACM SIGKDD International Conference on Knowledge Discovery in Data and Data Mining*, 2003.

[3] M. A. Aizerman, E. M. Braverman, and L. I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25:821–837, 1964.

[4] K. Beyer, J. Goldstein, R. Ramakrishnan, and U. Shaft. When is "nearest neighbor" meaningful? *Lecture Notes in Computer Science*, 1999.

[5] E. Y. Chang and et. al. PSVM: Parallel support vector machines. *Google Technical Report*, 2007.

[6] E. Y. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[7] J. Dean and S. Ghemawat. Mapreduce: Simplified data processing on large clusters. *OSDI'04: Symposium on Operating System Design and Implementation*, 2004.

[8] R. Fagin, R. Kumar, and D. Sivakumar. Efficient similarity search and classification via rank aggregation. In *Proceedings of ACM SIGMOD Conference on Management of Data*, pages 301–312, June 2003.

[9] A. Gionis, P. Indyk, and R. Motwani. Similarity search in high dimensions via hashing. In *Proceedings of the 25th VLDB Conference*, pages 518–529, 1999.

[10] K.-S. Goh, E. Y. Chang, and W.-C. Lai. Multimodal concept-depend ent active learning for image retrieval. *ACM Multimedia*, 2004.

[11] R. L. Goldstone. Similarity, interactive activation, and mapping. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 20:3–28, 1994.

[12] G. H. Golub and C. F. V. Loan. *Matrix Computations*. Johns Hopkins University Press, 1996.

[13] S. C. H. Hoi, M. R. Lyu, and E. Y. Chang. Learning the unified kernel machines for classification. *ACM KDD*, 2006.

[14] B. Li and E. Chang. Discovery of a perceptual distance function for measuring image similarity. *ACM Multimedia Journal Special Issue (to appear)*, 2002.

[15] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 2004.

[16] S. Mehrotra. On the implementation of a primal-dual interior point method. *SIAM J. Optimization*, 2:575–601, 1992.

[17] G. Murphy and D. Medin. The role of theories in conceptual coherence. *Psychological Review*, 92:289–316, 1985.

[18] N. Panda and E. Y. Chang. Efficient top-k hyperplane query processing for multimedia information retrieval. *ACM International Conference on Multimedia*, 2006.

[19] N. Panda and E. Y. Chang. Kdx: An indexer for support vector machines. *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, 18(6), 2006.

[20] N. Panda, E. Y. Chang, and G. Wu. Concept boundary detection for speeding up svms. *International Conference on Machine Learning (ICML)*, pages 681–688, 2006.

[21] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.

[22] A. K. H. Tung, R. Zhang, N. Koudas, and B. C. Ooi. Similarity searching: a matching based approach. In *Proceedings of the 32nd International Conference on Very La rge Databases*, pages 631–642, 2006.

[23] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.

[24] G. Wu and E. Y. Chang. Adaptive feature-space conformal transformation for learning imbala nced data. *International Conference on Machine Learning*, August 2003.

[25] G. Wu, E. Y. Chang, Y.-K. Chen, and C. Hughes. Incremental approximate matrix factorization for speeding up support vector machines. *ACM International Conference on Knowledge Discovery and Data Mining (KDD)*, 2006.

[26] G. Wu, E. Y. Chang, and N. Panda. Formulating context-dependent similarity functions. *ACM Multimedia*, pages 725–734, 2005.

[27] G. Wu, A. Rahimi, E. Y. Chang, K. Goh, T. Tsai, A. Jain, and Y.-F. Wang. Identifying color in motion in video sensors. *IEEE CVPR*, pages 561–569, 2006.

[28] G. Wu, Z. Zhang, and E. Y. Chang. Kronecker factorization for speeding up kernel machines. *SIAM International Conference on Data Mining (SDM)*, 2005.

[29] Y. Wu, E. Y. Chang, K. C.-C. Chang, and J. R. Smith. Optimal multimodal fusion for multimedia data analysis. *ACM International Conference on Multimedia (MM)*, pages 572–579, 2004.

[30] Y. Wu, E. Y. Chang, and B. Tseng. Multimodal metadata fusion using causal strength. *ACM International Conference on Multimedia (MM)*, pages 872–881, 2005.